

Web Cloud

Firebase - database

Cloud Firestore

Cloud Firestore est une base de données flexible et évolutive pour le développement mobile, Web et serveur à partir de Firebase et Google Cloud. Comme Firebase Realtime Database, il maintient vos données synchronisées entre les applications clientes via des écouteurs en temps réel et offre une prise en charge hors ligne pour mobile et Web afin que vous puissiez créer des applications réactives qui fonctionnent quelle que soit la latence du réseau ou la connectivité Internet. Cloud Firestore offre également une intégration transparente avec d'autres produits Firebase et Google Cloud, notamment Cloud Functions.

Créer des données

Notre application va être une application de blog, sur laquelle on peut voir tous les articles de blog et les commentaires déjà écrit, et ajouter un commentaire si on est authentifié

Configurer firestore

Direction la console de firebase > Créez > Firestore Database > Accéder à la console cloud > Mode natif

Configurer l'accès

Tout comme Cloud Storage, Firestore fournit un langage de règles déclaratives qui vous permet de définir comment vos données doivent être structurées, comment elles doivent être indexées et quand vos données peuvent être lues et écrites. Par défaut, l'accès en lecture et en écriture à Firestore est restreint afin que seuls les utilisateurs authentifiés puissent lire ou écrire des données.

Règle d'accès

On va changer la règle par défaut dans l'onglet Firestore Database > Règles pour lui donner l'autorisation en lecture sur tous les documents, et en écriture seulement si l'utilisateur est authentifié

```
rules_version = '2';
```

```
service cloud.firestore {
```

```
  match /databases/{database}/documents {
```

```
    match /{document=**} {
```

```
      allow read: if true;
```

```
      allow write: if request.auth != null;
```

```
    }
```

```
  }
```

```
}
```

Ajouter des données

Cloud Firestore stocke les données dans des documents, qui sont stockés dans des collections. Cloud Firestore crée implicitement des collections et des documents la première fois que vous ajoutez des données au document. Vous n'avez pas besoin de créer explicitement des collections ou des documents.

Exemple de code d'ajout de donnée

firebase/add_post_data.js

```
1  import app from "../firebaseConfig";
2  import { getFirestore } from "firebase/firestore";
3  import { collection, addDoc } from "firebase/firestore";
4  const db = getFirestore(app);
5
6  export const createPost = async (title, text, createdBy) => {
7    try {
8      const docRef = await addDoc(collection(db, "post"), {
9        title, text,
10       date: new Date(),
11       createdBy
12     });
13     console.log("Document written with ID: ", docRef.id);
14   } catch (e) {
15     console.error("Error adding document: ", e);
16   }
17 }
```


A vous de jouer

Créer une nouvelle page avec un formulaire pour pouvoir ajouter un post à votre blog. Attention seuls les personnes connectés peuvent ajouter le post

Récupérer la data

firebase/get_post_data.js

```
1  import app from "../firebaseConfig";
2  import { getFirestore } from "firebase/firestore";
3  import { collection, getDocs } from "firebase/firestore";
4  const db = getFirestore(app);
5
6  export const getPostData = async () => {
7    const querySnapshot = await getDocs(collection(db, "posts"));
8    let res = querySnapshot.docs.map((doc) => (
9      { id: doc.id, ...doc.data() }
10    ))
11    return res;
12  }
```

Afficher la data

Dans le fichier de la page d'accueil, vous allez pouvoir listez tous les posts créés

```
import React, {useState, useEffect} from "react";
import { StyleSheet, Text, View } from "react-native";
import { Link } from 'expo-router';
import { getPostData } from "../firebase/get_post_data";
export default function Home() {
  const [posts, setPosts] = useState([])
  useEffect(() => {
    const fetchData = async () => {
      const data = await getPostData();
      console.log(data);
      setPosts(data);
    }
    fetchData();
  }, [])
  return (
    <View style={styles.container}>
      <Text>Bienvenue</Text>
      <Link href="newpost">Créer un nouveau post</Link>
      {posts.map((p) => {
        return (
          <View key={p.id} style={styles.item}>
            <Text style={styles.itemTitle}>{p.title}</Text>
            <Text>{p.text}</Text>
          </View>
        );
      })}
    </View>
  );
}
```

A vous de jouer

Créer la fonctionnalité des commentaires :

Faire une page qui permet de voir un post en entier

Ajouter un bouton pour créer un commentaire

Ajouter une page de nouveau commentaire, accessible uniquement aux utilisateurs authentifiés

Afficher les commentaires sur la page du post, avec le nom de la personne ayant écrit le commentaire